Manejo de transacciones

Clase 22

IIC2413 - Sección 3

Prof. Cristian Riveros

Outline

Transacciones (anterior)

ACID (anterior)

Modelación de transacciones

Transacciones serializables

Outline

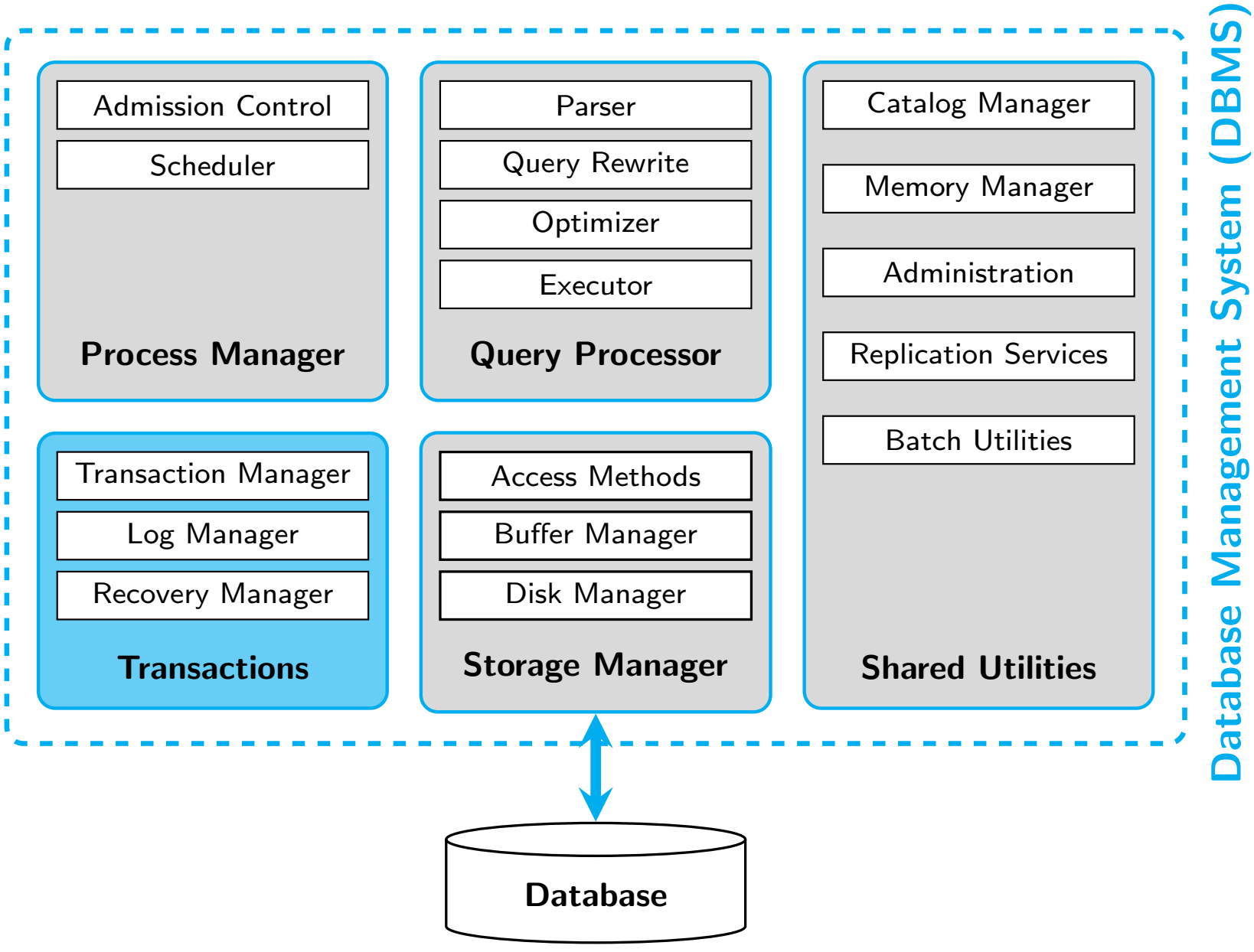
Transacciones (anterior)

ACID (anterior)

Modelación de transacciones

Transacciones serializables

Gestión de transacciones



Transacciones en SQL (recordatorio)

```
START TRANSACTION;
    <una-o-mas-consultas-o-actualizaciones>
COMMIT;
```

Ejemplo

```
START TRANSACTION;

    UPDATE Accounts
    SET balance = balance - 100
    WHERE id = '0001';

    UPDATE Accounts
    SET balance = balance + 100
    WHERE id = '0002';

COMMIT;
```

Permite agrupar operaciones **en una sola transacción**

Transacciones en SQL (recordatorio)

Importante

En ausencia de `START TRANSACTION ... COMMIT`
cada consulta es una transacción **por si sola**.

Ejemplo

```
UPDATE Accounts
SET balance = balance - 100
WHERE id = '0001';

UPDATE Accounts
SET balance = balance + 100
WHERE id = '0002';
```

≡

```
START TRANSACTION;
  UPDATE Accounts
  SET balance = balance - 100
  WHERE id = '0001';
COMMIT;
START TRANSACTION;
  UPDATE Accounts
  SET balance = balance + 100
  WHERE id = '0002';
COMMIT;
```

Transacciones en SQL: atomicidad (recordatorio)

¿cuál es el resultado de esta transacción?

Accounts

id	userId	balance	credit
0001	Alice	1.000	2.000
0002	Bob	1.000	1.000

```
CREATE TABLE Accounts (
   ...
   CHECK (balance + credit >= 0));
```

```
START TRANSACTION;

      UPDATE Accounts
      SET balance = balance + 3000
      WHERE id = '0001';

      UPDATE Accounts
      SET balance = balance - 3000
      WHERE id = '0002';

COMMIT;
```

Permite asegurar que cada transacción es **atómica**

(se hace entera o no se hace)

Transacciones en SQL: ROLLBACK

```
START TRANSACTION;
    <una-o-mas-consultas-o-actualizaciones>
ROLLBACK;
```

Ejemplo

Accounts

id	userId	balance	credit
0001	Alice	1.000	2.000
0002	Bob	1.000	1.000

Movements

time	aid	amount	comment
10:40	0001	+1000	Cajero
12:15	0002	+200	Deposito
13:30	0002	+800	Deposito

```
START TRANSACTION;

    UPDATE Accounts
    SET balance = balance + 100
    WHERE id = '0001';

ROLLBACK;
```

ROLLBACK **deshace los cambios** desde el inicio de la transacción

Transacciones en SQL: SAVEPOINT

```
SAVEPOINT <nombre-savepoint>;
<una-o-mas-consultas-o-actualizaciones>
ROLLBACK TO SAVEPOINT <nombre-savepoint>;
```

Ejemplo

```
START TRANSACTION;
      UPDATE Accounts
      SET balance = balance - 100
      WHERE id = '0001';
      SAVEPOINT half-transaction;
      UPDATE Accounts
      SET balance = balance + 100
      WHERE id = '0002';
      ROLLBACK TO SAVEPOINT half-transaction;
COMMIT;
```

SAVEPOINT permite hacer ROLLBACK **hasta algún punto** de la transacción

Transacciones en SQL: restricciones de integridad

Por defecto, durante una transacción
las restricciones son verificadas después de cada operación.

¿cuál es el resultado de esta transacción?

Accounts

id	userId	balance	credit
0001	Alice	1.000	2.000
0002	Bob	1.000	1.000

Movements

time	aid	amount	comment
10:40	0001	+1000	Cajero
12:15	0002	+200	Deposito
13:30	0002	+800	Deposito

```
START TRANSACTION;
      INSERT INTO Accounts
      VALUES ('0001', 'Ana', 1000, 500);
      DELETE FROM Accounts
      WHERE id = '0001' AND userId = 'Alice';
COMMIT;
```

En algunos casos, es necesario **levantar las restricciones** momentáneamente

Transacciones en SQL: DEFERRABLE

Ejemplo

```
CREATE TABLE Accounts (
   id CHAR(4) PRIMARY KEY,
   userid VARCHAR(20),
   ...
   CONSTRAINT uniqueuserid UNIQUE (userid)
   DEFERRABLE INITIALLY IMMEDIATE);


START TRANSACTION;
      SET CONSTRAINTS uniqueuserid DEFERRED;
      INSERT INTO Accounts
      VALUES ('0001', 'Ana', 1000, 500);
      DELETE FROM Accounts
      WHERE id = '0001' AND userId = 'Alice';
COMMIT;
```

DEFFERABLE permite **diferir la restricción** hasta el COMMIT

Transacciones en SQL: DEFERRABLE

Ejemplo (siempre diferir)

```
CREATE TABLE Accounts (
  id CHAR(4) PRIMARY KEY,
  userid VARCHAR(20),
  ...
  CONSTRAINT uniqueuserid UNIQUE (userid)
  DEFERRABLE INITIALLY DEFERRED);


START TRANSACTION;
      INSERT INTO Accounts
      VALUES ('0001', 'Ana', 1000, 500);
      DELETE FROM Accounts
      WHERE id = '0001' AND userId = 'Alice';
COMMIT;
```

`INITIALLY DEFERRED` no requiere usar `SET CONSTRAINTS ... DEFERRED`

Sobre transacciones

- Unos de los componentes principales de una base de datos.
- Fundamental para muchas aplicaciones.
- Uno de los premios **Turing Award** en BD:

Charles Bachman	1973	primeros cimientos para DBMS.
Edgar Codd	1981	por inventar el modelo relacional.
Jim Gray	1998	por inventar las **transacciones**.
Michael Stonebracker	2015	por desarrollar Ingres.

Outline

Transacciones (anterior)

ACID (anterior)

Modelación de transacciones

Transacciones serializables

Garantías ACID

ACID = **A**tomicity
Consistency
Isolation
Durability

Todos los sistemas de bases de datos **deben garantizar ACID**

(pero no todos lo hacen)

Garantías ACID

Atomicity

Se ejecutan todos los pasos de una transacción o no se ejecuta nada.

Ejemplo

```
START TRANSACTION;

      UPDATE Accounts
      SET balance = balance - 100
      WHERE id = '0001';

      UPDATE Accounts
      SET balance = balance + 100
      WHERE id = '0002';

COMMIT;
```

¿qué pasa si justo **falla el sistema** después del primer update?

Se ejecutan ambos updates o no se ejecuta ninguno.

Garantías ACID

Atomicity

Se ejecutan todos los pasos de una transacción o no se ejecuta nada.

Consistency

Al terminar una transacción los datos deben ser
correctos y consistentes según las restricciones de integridad.

Garantías ACID

Consistency

Al terminar una transacción los datos deben ser correctos y consistentes según las restricciones de integridad.

Ejemplo

Accounts

id	userId	balance	credit
0001	Alice	1.000	2.000
0002	Bob	1.000	1.000

```
CREATE TABLE Accounts (
   ...
   CHECK (balance + credit >= 0));
```

```
START TRANSACTION;
      UPDATE Accounts SET balance = balance + 3000
      WHERE id = '0001';
      UPDATE Accounts SET balance = balance - 3000
      WHERE id = '0002';
COMMIT;
```

Si el estado final no es consistente, la transacción fallará.

Garantías ACID

Atomicity

Se ejecutan todos los pasos de una transacción o no se ejecuta nada.

Consistency

Al terminar una transacción los datos deben ser
correctos y consistentes según las restricciones de integridad.

Isolation

Cada transacción se ejecuta sin ser interferida por otras transacciones.

Garantías ACID

Isolation

Cada transacción se ejecuta sin ser interferida por otras transacciones.

Ejemplo: traspaso de dinero entre cuentas

Alice	**Bob**	b_1	b_2	
`READ`(b_1, x)		1000	1000	
`WRITE`$(b_1, x-100)$		900		
`READ`(b_2, x)				
	`READ`(b_1, y)			
	`WRITE`$(b_1, y-100)$	800		
	`READ`(b_2, y)			
	`WRITE`$(b_2, y+100)$		1100	
`WRITE`$(b_2, x+100)$		800	1100	✕

Una transacción no puede interferir con otra transacción

(pero queremos paralelizar lo más posible)

Garantías ACID

Atomicity

Se ejecutan todos los pasos de una transacción o no se ejecuta nada.

Consistency

Al terminar una transacción los datos deben ser
correctos y consistentes según las restricciones de integridad.

Isolation

Cada transacción se ejecuta sin ser interferida por otras transacciones.

Durability

Si un transacción hace commit,
sus cambios sobrevivirán cualquier tipo de falla.

Garantías ACID

Durability

Si un transacción hace commit,
sus cambios sobrevivirán cualquier tipo de falla.

Ejemplo: traspaso de dinero entre cuentas

Alice	**Bob**	b_1	b_2
`READ`(b_1, x)		1000	1000
`WRITE`$(b_1, x-100)$		900	
`READ`(b_2, x)			
`WRITE`$(b_2, x+100)$			1000
	`READ`(b_1, y)		
	`WRITE`$(b_1, y-100)$	800	
	`READ`(b_2, y)		
	`WRITE`$(b_2, y+100)$	800	1200
`ERROR`	`ERROR`	(datos en buffer)	



Una vez que haya un `COMMIT` exitoso, se persisten los cambios

¿cuándo NO tenemos ACID?

Atomicity

✗ Una transacción se ejecuta solo a medias
pero afecta el estado de la base de datos.

Consistency

✗ Al ejecutar la transacción,
la base de datos no satisface las restricciones de integridad.

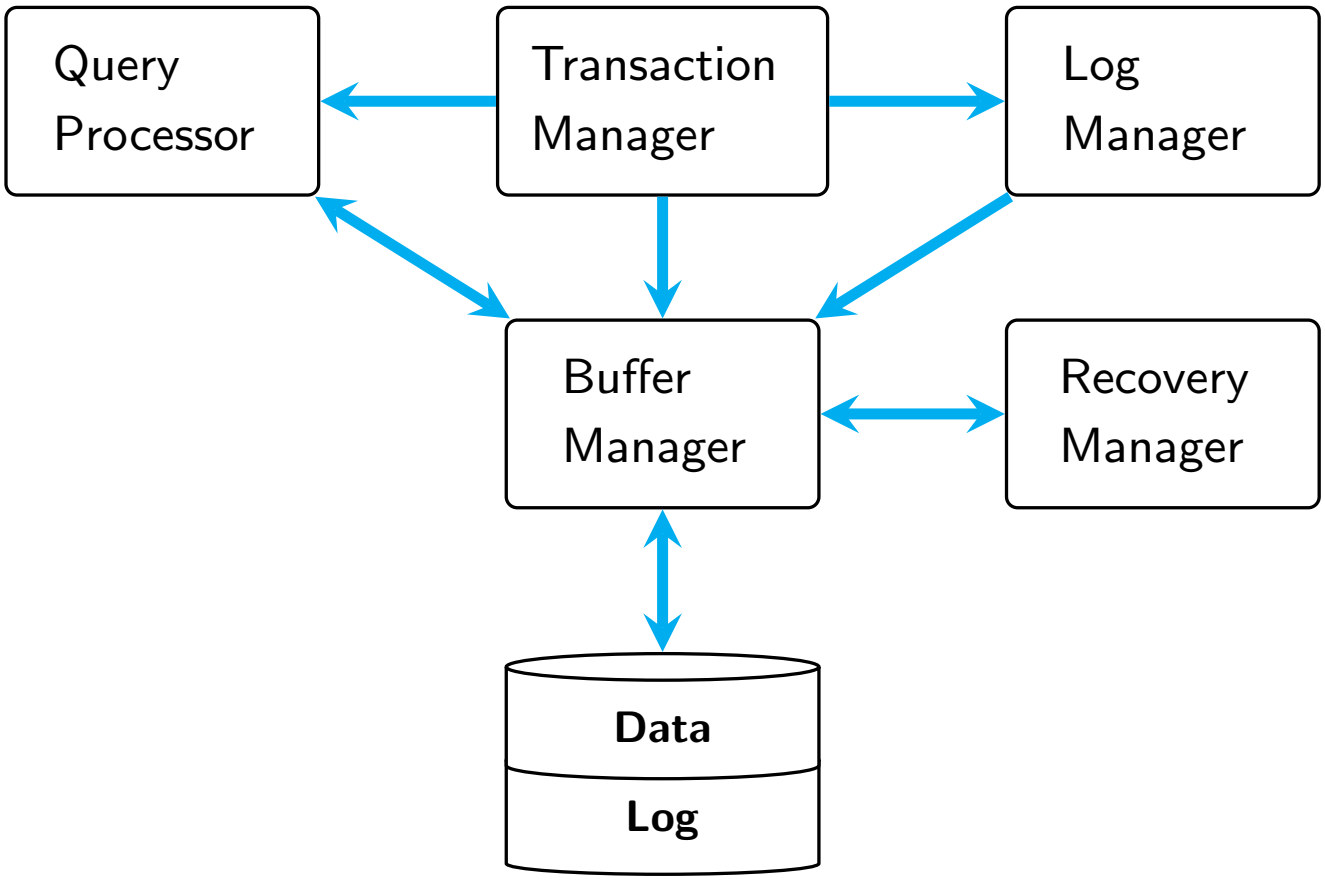
Isolation

✗ El resultado final de dos transacciones no es
equivalente a ejecutar cada transacción en serie.

Durability

✗ La base de datos se actualiza momentáneamente y
luego vuelve al estado anterior.

¿cómo podemos asegurar las propiedades ACID?



¿cómo podemos asegurar las propiedades ACID?

¿qué puede salir mal?

1. Transacciones concurrentes
   - Isolation
   - Consistency

   Encargado: **Transaction** Manager

2. Recuperación a fallas
   - Atomicity
   - Durability

   Encargado: **Log** Manager y el **Recovery** Manager

Outline

Transacciones (anterior)

ACID (anterior)

Modelación de transacciones

Transacciones serializables

Transaction manager

Isolation (desiderata)

"Si bien las acciones de varias transacciones pueden ser intercaladas o ocurrir en paralelo, el resultado final es idéntico a ejecutar todas las transacciones una después de la otra en **algún** *orden secuencial."*

¿cómo formalizamos **qué es una transacción**?

Transacciones: abstracción

Definición

- Una **transacción** es una secuencia de una o más operaciones que leen o modifican la bases de datos.
- Cada transacción esta compuesta por:
  1. una secuencia de **instrucciones**.
  2. un **estado**: posición en las instrucciones y variables temporales.

Ejemplo de una transacción

Estado

Instrucciones

Transacción	x	B_1	B_2
`READ`(B_1, x)	800	800	500
$x := x - 100$	700		
`WRITE`(B_1, x)		700	
`READ`(B_2, x)	500		
$x := x * 3$	1500		
`WRITE`(B_2, x)			1500

Transacciones: elementos (datos)

Definición

Un **elemento** de una base de datos es un valor o conjunto de valores que puede ser leído o modificado por una transacción.

- dato
- tupla
- página
- relación

Ejemplo de elementos

		Elementos	
Transacción	x	B_1	B_2
READ(B_1, x)	800	800	500
$x := x - 100$	700		
WRITE(B_1, x)		700	
READ(B_2, x)	500		
$x := x * 3$	1500		
WRITE(B_2, x)			1500

Transacciones: elementos (datos)

Definición

Un **elemento** de una base de datos es un valor o conjunto de valores que puede ser leído o modificado por una transacción.

- dato
- tupla
- página
- relación

¿qué **granularidad** de elemento es preferible?

Transacciones: estado y correctitud

Definición

- El **estado** de una base de datos es
  el valor para cada uno de sus elementos.
- El estado de una base de datos es **consistente** si satisface:
  - restricciones de integridad.
  - restricciones **implícitas**.

Principio de correctitud de transacciones

"Si una transacción **comienza con un estado consistente** *de la base de datos y se ejecuta en ausencia de otras transacciones o errores de sistema, entonces la transacción* **terminará con un estado consistente** *de la base de datos."*

¿és **razonable**? ¿qué sucede con las restricciones **implicitas**?

Outline

Transacciones (anterior)

ACID (anterior)

Modelación de transacciones

Transacciones serializables

Transaction manager

Isolation (desiderata)

"Si bien las acciones de varias transacciones pueden ser intercaladas o ocurrir en paralelo, el resultado final es idéntico a ejecutar todas las transacciones una después de la otra en **algún** *orden secuencial."*

¿cómo formalizamos **esta propiedad**?

Schedules, serial y serializable

Definición

- Un **schedule** es una secuencia de **instrucciones primitivas** de una o más transacciones $T_1, \ldots, T_n$ tal que cada T_i aparecen en el mismo orden de su definición.

Ejemplo de dos transacciones

$\mathbf{T_1}$	$\mathbf{T_2}$
$\texttt{READ}(A, x)$	$\texttt{READ}(A, y)$
$x := x + 100$	$y := y * 2$
$\texttt{WRITE}(A, x)$	$\texttt{WRITE}(A, y)$
$\texttt{READ}(B, x)$	$\texttt{READ}(B, y)$
$x := x + 200$	$y := y * 3$
$\texttt{WRITE}(B, x)$	$\texttt{WRITE}(B, y)$

Schedules, serial y serializable

Ejemplo de un schedule para $\mathbf{T_1}$ y $\mathbf{T_2}$

$\mathbf{T_1}$	$\mathbf{T_2}$
READ(A, x)	
$x := x + 100$	
WRITE(A, x)	
READ(B, x)	
$x := x + 200$	
WRITE(B, x)	
	READ(A, y)
	$y := y * 2$
	WRITE(A, y)
	READ(B, y)
	$y := y * 3$
	WRITE(B, y)

Schedules, serial y serializable

Ejemplo de un schedule para $\mathbf{T_1}$ y $\mathbf{T_2}$ (2)

$\mathbf{T_1}$	$\mathbf{T_2}$
READ(A, x)	
$x := x + 100$	
WRITE(A, x)	
	READ(A, y)
	$y := y * 2$
	WRITE(A, y)
READ(B, x)	
$x := x + 200$	
WRITE(B, x)	
	READ(B, y)
	$y := y * 3$
	WRITE(B, y)

Schedules, serial y serializable

Definición

- Un **schedule** es una secuencia de **instrucciones primitivas** de una o más transacciones $T_1, \ldots, T_n$ tal que cada T_i aparecen en el mismo orden de su definición.
- Un schedule es **serial** si todas las acciones de cada transacción son ejecutas en grupo sin intercalación de acciones.

Schedules, serial y serializable

¿cuál schedule es serial?

$\mathbf{T_1}$	$\mathbf{T_2}$	A	B
READ(A, x)		100	50
$x := x + 100$			
WRITE(A, x)		200	
READ(B, x)			
$x := x + 200$			
WRITE(B, x)			250
	READ(A, y)		
	$y := y * 2$		
	WRITE(A, y)	400	
	READ(B, y)		
	$y := y * 3$		
	WRITE(B, y)		750

Schedules, serial y serializable

¿cuál schedule es serial? (2)

$\mathbf{T_1}$	$\mathbf{T_2}$	A	B
	READ(A, y)	100	50
	$y := y * 2$		
	WRITE(A, y)	200	
	READ(B, y)		
	$y := y * 3$		
	WRITE(B, y)		150
READ(A, x)			
$x := x + 100$			
WRITE(A, x)		300	
READ(B, x)			
$x := x + 200$			
WRITE(B, x)			350

Schedules, serial y serializable

¿cuál schedule es serial? (3)

$\mathbf{T_1}$	$\mathbf{T_2}$	A	B
READ(A, x)		100	50
$x := x + 100$			
WRITE(A, x)		200	
	READ(A, y)		
	$y := y * 2$		
	WRITE(A, y)	400	
READ(B, x)			
$x := x + 200$			
WRITE(B, x)			250
	READ(B, y)		
	$y := y * 3$		
	WRITE(B, y)		750

Schedules, serial y serializable

Definición

- Un **schedule** es una secuencia de **instrucciones primitivas** de una o más transacciones $T_1, \ldots, T_n$ tal que cada T_i aparecen en el mismo orden de su definición.
- Un schedule es **serial** si todas las acciones de cada transacción son ejecutas en grupo sin intercalación de acciones.
- Un schedule S es **serializable** si existe algún schedule serial S' tal que el resultado de S y S' es el mismo para todo estado inicial de la BD.

Schedules, serial y serializable

¿cuál schedule es serializable?

$\mathbf{T_1}$	$\mathbf{T_2}$	A	B
READ(A, x)		100	50
$x := x + 100$			
WRITE(A, x)		200	
	READ(A, y)		
	$y := y * 2$		
	WRITE(A, y)	400	
READ(B, x)			
$x := x + 200$			
WRITE(B, x)			250
	READ(B, y)		
	$y := y * 3$		
	WRITE(B, y)		750

Schedules, serial y serializable

¿cuál schedule es serializable?

$\mathbf{T_1}$	$\mathbf{T_2}$	A	B
READ(A, x)		100	50
$x := x + 100$			
WRITE(A, x)		200	
	READ(A, y)		
	$y := y * 2$		
	WRITE(A, y)	400	
READ(B, x)			
$x := x + 200$			
WRITE(B, x)			250
	READ(B, y)		
	$y := y * 3$		
	WRITE(B, y)		750

Schedules, serial y serializable

¿cuál schedule es serializable? (2)

$\mathbf{T_1}$	$\mathbf{T_2}$	A	B
READ(A, x)		100	50
$x := x + 100$			
WRITE(A, x)		200	
	READ(A, y)		
	$y := y * 2$		
	WRITE(A, y)	400	
	READ(B, y)		
	$y := y * 3$		
	WRITE(B, y)		150
READ(B, x)			
$x := x + 200$			
WRITE(B, x)			350

Schedules, serial y serializable

Definición

- Un **schedule** es una secuencia de **instrucciones primitivas** de una o más transacciones $T_1, \ldots, T_n$ tal que cada T_i aparecen en el mismo orden de su definición.
- Un schedule es **serial** si todas las acciones de cada transacción son ejecutas en grupo sin intercalación de acciones.
- Un schedule S es **serializable** si existe algún schedule serial S' tal que el resultado de S y S' es el mismo para todo estado inicial de la BD.

La tarea del Transaccion Manager
es permitir solo schedules que son **serializables**

¿és posible verificar que un schedule es serializable?

Ejemplo de schedule no serializable

$\mathbf{T_1}$	$\mathbf{T_2}$	A	B
READ(A, x)		100	50
$x := x + 100$			
WRITE(A, x)		200	
	READ(A, y)		
	$y := y * 2$		
	WRITE(A, y)	400	
	READ(B, y)		
	$y := y * 3$		
	WRITE(B, y)		150
READ(B, x)			
$x := x + 200$			
WRITE(B, x)			350

¿és posible verificar que un schedule es serializable?

Ejemplo de schedule serializable

T_1	T_2	A	B
READ(A, x)		100	50
$x := x + 100$			
WRITE(A, x)		200	
	READ(A, y)		
	$y := y + 300$		
	WRITE(A, y)	500	
	READ(B, y)		
	$y := y + 400$		
	WRITE(B, y)		450
READ(B, x)			
$x := x + 200$			
WRITE(B, x)			650

¡solo cambiamos dos operaciones!

¿és posible verificar que un schedule es serializable?

Conclusiones

- En general, es muy difícil verificar si un schedule es serializable.
- Simplificamos el análisis
  considerando solo los `READ` y `WRITE` de un objeto.

Simplificación

Desde ahora, las instrucciones de las transacciones
solo tendrán operaciones primitivas `READ` y `WRITE` de un objeto:

- $T_i : \texttt{READ}(X, t) \;\Rightarrow\; r_i(X)$
- $T_i : \texttt{WRITE}(X, t) \;\Rightarrow\; w_i(X)$

¿qué tan **razonable** es esta simplificación?

¿és posible verificar que un schedule es serializable?

Ejemplo de simplificación

Antes	**Ahora**
T_1 : READ(A, x)	$r_1(A)$
T_1 : $x := x + 100$	
T_1 : WRITE(A, x)	$w_1(A)$
T_2 : READ(A, y)	$r_2(A)$
T_2 : $y := y * 2$	
T_2 : WRITE(A, y)	$w_2(A)$
T_2 : READ(B, y)	$r_2(B)$
T_2 : $y := y * 3$	
T_2 : WRITE(B, y)	$w_2(B)$
T_1 : READ(B, x)	$r_1(B)$
T_1 : $x := x + 200$	
T_1 : WRITE(B, x)	$w_1(B)$

En línea: $r_1(A)\ w_1(A)\ r_2(A)\ w_2(A)\ r_2(B)\ w_2(B)\ r_1(B)\ w_1(B)$